Retail AI Assistant — Architecture Document

Overview

This project implements a single agentic AI system that simulates two roles:

- Personal Shopper (Revenue Agent) — recommends products based on user constraints
- Customer Support Assistant (Operations Agent) — evaluates return eligibility using policy rules

The system follows a tool-based architecture, where:

- The LLM does NOT access raw data directly
- All data comes from structured tools
- The LLM only:
    1. Selects the correct tool
    2. Explains results using tool output

This ensures zero hallucination and high reliability.

System Architecture

Core Components

1. main.py (CLI Layer)

- Handles user interaction
- Accepts **free-text input** (no restriction on format)
- Routes input to the agent
- Supports both:
    - Product queries
    - Return/support queries

2. agent/agent.py (Agent Orchestrator)

This is the brain of the system, responsible for:

a. Tool Selection (LLM-based)

- Uses SYSTEM_PROMPT + SEARCH_PRODUCTS_GUIDE
- Converts natural language → structured JSON:

{"tool": "search_products", "args": {...}}

b. Validation Layer

- Ensures:

    - Tool name is allowed
    - Arguments are valid
- Prevents invalid or hallucinated calls

c. Tool Execution

- Calls execute_tool() from main.py
- Retrieves real data from CSVs

d. Output Validation

- Verifies tool output matches strict schema
- Rejects unexpected or malformed data

e. Final Response Generation

- Uses FINAL_RESPONSE_PROMPT
- Enforces:
    - Business reasoning
    - No hallucination
    - Evidence-based explanation

f. Fallback Logic

- If LLM fails:
    - Uses parse_shopper_query() (rule-based extraction)
    - Uses deterministic_final_response()
- Ensures system never breaks

3. tools/ (Data & Business Logic Layer)

All tools operate on CSV data only:

search_products(filters)

- Filters by:
    - price
    - size
    - sale status
    - tags
- Checks:
    - sizes_available
    - stock_per_size

get_product(product_id)

- Returns full product details

get_order(order_id)

- Fetches order information

evaluate_return(order_id)

- Applies **policy rules**
- Returns:
  - eligibility
  - outcome
  - explanation
  - evidence

4. utils/data_loader.py

- Loads:
  - Products CSV
  - Orders CSV
  - Policy file
- Converts policy into structured dictionary

Data Flow

User Input
↓
main.py (CLI)
↓
agent.py
↓
LLM → Tool Selection (JSON)
↓
Validation Layer
↓
execute_tool()
↓
tools/ (CSV data access)
↓
Structured Tool Output
↓
agent.py (Final Prompt)
↓
LLM → Explanation
↓
Final Response (with reasoning + evidence)

How Hallucination is Prevented

1. Tool-Only Data Access

- LLM cannot access CSV directly
- Must call tools for any data

2. Strict Content Schema

Defined in CONTENT_SCHEMA:

- Forces model to use only:
  - price
  - sizes_available
  - stock_per_size
  - etc.

No extra fields allowed.

3. Validation Layer

- Every tool output is checked before use
- Invalid data is rejected

4. Prompt Constraints

FINAL_RESPONSE_PROMPT enforces:

- Use ONLY tool data
- No assumptions
- No invented policies

5. Evidence Section (Critical Feature)

Every response includes:

Agent Work:

- Tool used
- Exact fields used

Model Role:

- Confirms model only formatted tool output

This makes hallucination immediately visible.

6. Deterministic Fallback

- Rule-based parsing + response generation
- Works without LLM
- Ensures reliability during demos

Tool Selection Strategy

Primary (LLM-based)

- LLM maps user query → tool + arguments
- Example:

"I need a gown under $300 in size 8"
→ search_products(max_price=300, size="8")

Fallback (Rule-based)

- Regex extraction:
  - price → under $300
  - size → size 8
  - tags → modest, evening
- Ensures:
  - System still works if LLM fails

Business Logic Implementation

Product Recommendation

The agent:

- Filters by:
  - price
  - size
  - stock availability
- Prioritizes:
  - is_sale = true
  - higher bestseller_score
- Selects best product (not list)

Return Evaluation

The system applies rules in order:

1. **Clearance → No return**
2. **Sale items → shorter return window**
3. **Vendor rules → overrides**
4. **Standard return window**

Output includes:

- decision (YES/NO)
- explanation
- evidence (days_passed, product_id)

Why This Architecture Works

1. Clear Separation of Concerns

- Tools → Data & logic
- Agent → Decision making
- LLM → Language generation

2. High Reliability

- No hallucinated data
- Always grounded in CSV + policy

3. Real Agent Behavior

- Dynamic tool selection
- Multi-step reasoning
- Business-aware decisions

4. Production-Like Design

- Validation layers
- Structured outputs
- Fallback mechanisms

Conclusion

This system successfully demonstrates a tool-based agent architecture that:

- Handles both shopping and support tasks
- Uses real data only
- Performs multi-constraint reasoning
- Applies policy-based decisions
- Maintains zero hallucination